SG-100 Write Logic -- Configuration Register Communication

This document describes the complete write communication pipeline used by the Android HT-SG100 companion application to modify configuration parameters on the Huegli Tech SG-100 Speed Governor. It covers packet construction, USB HID transport, write-enable sequencing, response handling, CRC verification, error recovery, and UI integration.

Relevant code paths:

* DashboardViewModel.writeRegister() receives user intent and coordinates the write sequence.
* RegisterRepository.writeSingleRegister() constructs packets, executes the write-enable gate, transmits the write, and verifies the response.
* ModbusPacketBuilder.writeMultipleRegisters() constructs FC16 Modbus RTU frames.
* UsbHidManager.exclusiveWrite() stops background polling workers and performs a direct low-level USB HID write with echo capture.
* RegisterDecoder.decode() verifies CRC and dispatches the device echo response.
* SettingsManager propagates validated written values back into the UI state.

---

1. Introduction

1.1 Purpose

The SG-100 Speed Governor stores all operational configuration in non-volatile holding registers. These include speed setpoints, PID gains, actuator current limits, timing parameters, and hardware configuration flags. The Android companion application provides a field-configurable interface to modify these parameters without requiring a Windows PC running the Huegli Tech desktop tool.

Write operations allow an engineer or technician to:

* Adjust speed setpoints (Speed 1, Speed 2, Speed 3, Idle Speed, Overspeed)
* Tune PID control loop gains (P, I, D for both speed and position loops)
* Configure ramp times, current limits, PWM frequency, and droop percentage
* Set hardware flags for actuator mode, synchronisation, and J1939 bus options

1.2 Write Operations vs Read Operations

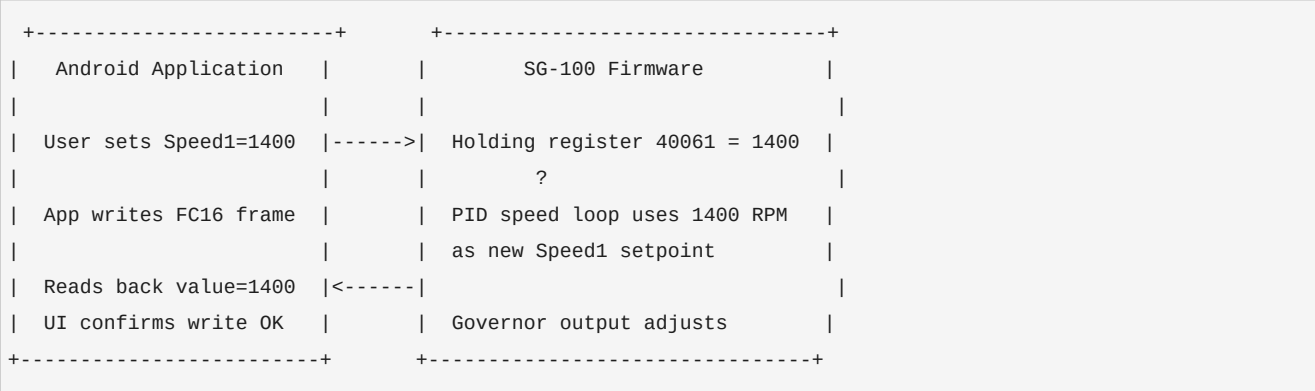
Read operations (FC04, FC03) are non-destructive and are performed continuously by the background polling loop at 10 Hz. Write operations are disruptive: they require stopping the polling bus, transmitting a configuration change, waiting for the device echo, and optionally reading back the updated register block.

The application coordinates reads and writes through an exclusive access mechanism that prevents a polling read response from landing in the receive

buffer during the write echo window.

1.3 Relationship Between Application and SG-100 Firmware

The companion application is a configuration interface, not a control surface. It modifies the SG-100's persistent holding registers. The SG-100 firmware reads these registers and applies them immediately to its internal PID control loop. The application does not directly control the governor output; it only changes the parameters the governor uses.

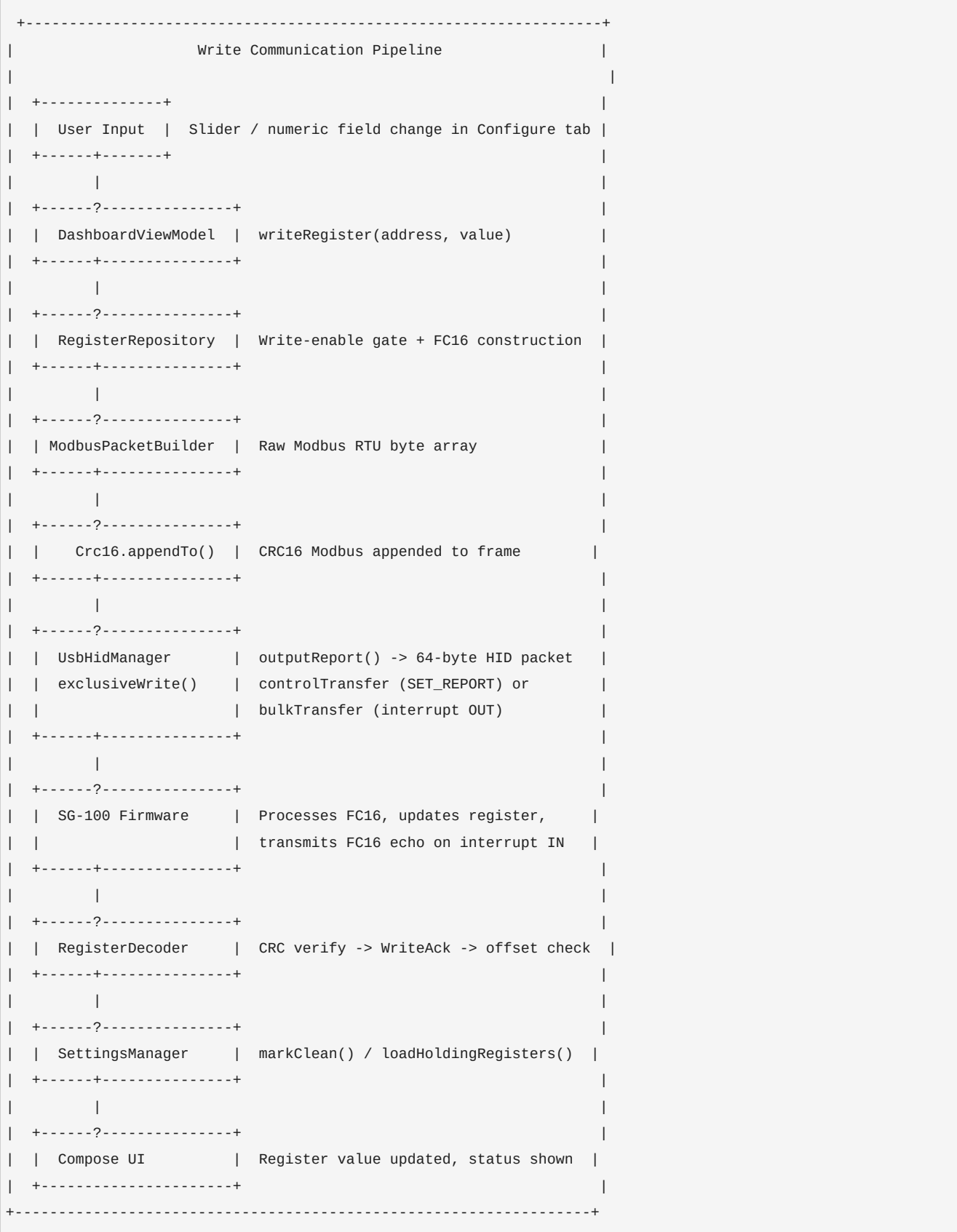


The firmware applies updated register values immediately. There is no explicit "commit" or "save" command required after individual register writes.

---

2. Write Communication Architecture

The complete write pipeline involves seven stages, from user interaction through to UI confirmation.



3. Write Packet Structure

3.1 Modbus RTU Frame -- FC16 Write Multiple Registers

All configuration writes use **Function Code 0x10 (FC16 -- Write Multiple Registers)** with a register count of 1. FC06 (Write Single Register) was investigated and rejected because the SG-100 firmware silently discards FC06 requests to holding registers; only FC16 produces a valid echo and register update.

```
 FC16 Write Request Frame

Byte 0      Slave ID             0x01  (SG-100 always slave 1)
Byte 1      Function Code        0x10  (Write Multiple Registers)
Byte 2      Start Addr High      High byte of register wire offset
Byte 3      Start Addr Low       Low byte of register wire offset
Byte 4      Register Count High  0x00  (always 0 for single-register writes)
Byte 5      Register Count Low   0x01  (writing one register)
Byte 6      Byte Count           0x02  (two data bytes for one 16-bit register)
Byte 7      Data High Byte       High byte of value to write
Byte 8      Data Low Byte        Low byte of value to write
Byte 9      CRC Low              Low byte of Modbus CRC16
Byte 10     CRC High             High byte of Modbus CRC16
```

Total Modbus RTU frame: **11 bytes**.

3.2 Register Address Encoding

The SG-100 uses display addresses in the 40001-49999 range for holding registers. The wire offset transmitted in the packet is the display address minus the base address 40001.

```
 Wire offset = Display address ? 40001

Examples:
  40051  ->  50  =  0x0032    (Overspeed)
  40061  ->  60  =  0x003C    (Speed 1)
  40067  ->  66  =  0x0042    (Speed PID I1)
  40068  ->  67  =  0x0043    (Speed PID P1)
  40079  ->  78  =  0x004E    (Write-enable gate register)
```

3.3 Example: Write Speed 1 = 1400 RPM

```
 Display address: 40061
Wire offset:     60 = 0x003C
Value:           1400 = 0x0578

Byte 0:  01    Slave ID
Byte 1:  10    FC16
Byte 2:  00    Offset high
Byte 3:  3C    Offset low (0x003C = 60 -> register 40061)
Byte 4:  00    Count high
Byte 5:  01    Count low (1 register)
Byte 6:  02    Byte count (2 data bytes)
Byte 7:  05    Data high (0x0578 high byte)
Byte 8:  78    Data low  (0x0578 low byte)
Byte 9:  CRC_L
Byte 10: CRC_H
```

Full packet (with CRC computed):

```
01 10 00 3C 00 01 02 05 78 C5 46
```

3.4 FC16 Echo Response

The SG-100 acknowledges a successful FC16 write by echoing the slave ID, function code, starting address, and register count. It does **not** echo the written data value.

```
 FC16 Echo Response

Byte 0      Slave ID             0x01
Byte 1      Function Code        0x10
Byte 2      Start Addr High      Mirrors request bytes 2-3
Byte 3      Start Addr Low
Byte 4      Register Count High  0x00
Byte 5      Register Count Low   0x01
Byte 6      CRC Low
Byte 7      CRC High
```

Total: **8 bytes**.

Example echo for the Speed 1 write above:

```
01 10 00 3C 00 01 A1 C8
```

3.5 Write-Enable Gate Frame

The SG-100 firmware requires a write-enable command before accepting configuration writes. This was determined by reverse-engineering the IL bytecode of the Windows PC configuration tool (SpeedGovernorConfiguration.exe). The firmware disables the write-enable flag after processing the configuration write or after a timeout.

Write-enable is performed by sending an FC16 write of value 0x0080 (128) to register 40079 (wire offset 78 = 0x004E):

```
01 10 00 4E 00 01 02 00 80 4C 03
```

Byte breakdown:

```
 01   Slave ID
10   FC16
00   Offset high
4E   Offset low (0x004E = 78 -> register 40079)
00   Count high
01   Count low
02   Byte count
00   Data high (0x0080 high byte)
80   Data low  (value 128)
4C   CRC low
03   CRC high
```

This write-enable packet is transmitted as a separate exclusive-write operation immediately before the main configuration write.

3.6 HID Report Encapsulation

Modbus RTU frames are encapsulated in 64-byte USB HID output reports before transmission. The SG-100 uses a fixed 64-byte report size. Unused bytes are zero-padded.

```
 HID Output Report (64 bytes)

Byte  0:   Modbus byte 0    (Slave ID: 0x01)
Byte  1:   Modbus byte 1    (Function: 0x10)
Byte  2:   Modbus byte 2    (Start Addr High)
Byte  3:   Modbus byte 3    (Start Addr Low)
Byte  4:   Modbus byte 4    (Count High: 0x00)
Byte  5:   Modbus byte 5    (Count Low: 0x01)
Byte  6:   Modbus byte 6    (Byte Count: 0x02)
Byte  7:   Modbus byte 7    (Data High)
Byte  8:   Modbus byte 8    (Data Low)
Byte  9:   Modbus byte 9    (CRC Low)
Byte 10:   Modbus byte 10   (CRC High)
Bytes 11-63: 0x00           (Zero padding)
```

Critical implementation note: USB HID SET_REPORT control transfers require that the 64-byte report data begins directly with the Modbus slave ID byte. A common mistake is to prepend a 0x00 report-ID byte, causing the device to receive 0x00 as the slave ID and reject the request with a USB STALL response. The report ID is communicated via the wValue field of the HID SET_REPORT control transfer header, not in the data payload.

```
 Correct HID output report (no report-ID prefix in data):
  01 10 00 3C 00 01 02 05 78 C5 46 00 00 00 ... 00

Incorrect (causes USB STALL):
  00 01 10 00 3C 00 01 02 05 78 C5 46 00 00 ... 00
  ^^ 0x00 report-ID byte prepended -- device rejects
```

---

4. Register Write Process

4.1 Step-by-Step Flow

```
 Step 1  User edits a parameter in the Configure tab
        ?
Step 2  SettingsManager.edit() clamps the value to [min, max]
        and marks the register as dirty
        ?
Step 3  User taps Write
        ?
Step 4  DashboardViewModel.writeRegister(address, value) called
        SettingsManager.markPending(address) -- UI shows "pending" state
        PollingManager.pause() -- background FC04/FC03 workers cancelled
        ?
Step 5  RegisterRepository.writeSingleRegister(address, value) called
        ?
Step 6  Write-enable gate:
        Build FC16 packet for register 40079, value 0x80
        UsbHidManager.exclusiveWrite(writeEnableTx)
        Workers are stopped; exclusive access to USB bus
        ?
Step 7  200 ms settle delay
        ?
Step 8  Main configuration write:
        ModbusPacketBuilder.writeMultipleRegisters(1, address, [value])
        -> FC16 Modbus RTU frame constructed
        -> Crc16.appendTo() appends CRC16
        -> outputReport() pads to 64 bytes
        UsbHidManager.exclusiveWrite(fc16Tx)
        ?
Step 9  Echo verification:
        Read interrupt IN endpoint until FC16 echo received or timeout
        RegisterDecoder.decode(rx) -> ModbusDecodeResult.WriteAck
        Verify ack.registerOffset == expected offset
        If verified: echoVerified = true -> return WriteResult.Success
        ?
Step 10 If no echo (timeout):
        200 ms settle delay
        FC03 readback: readHoldingRegisters() via normal exchange()
        Decode readback -> extract register value
        Return WriteResult.SuccessWithHolding(block)
        ?
Step 11 PollingManager.resume() -- background workers restarted
        ?
Step 12 UI update:
        SettingsManager.markClean(address, actualValue)
        SettingsManager.loadHoldingRegisters(block.registers)
        UI shows confirmed value, write status cleared
```

4.2 Worker Exclusion During Write

Background polling transmits FC04 read requests continuously at 10 Hz. If an FC04 response arrives in the receive buffer during the write echo window, the echo reader will consume the wrong packet and either misinterpret it or time out.

UsbHidManager.exclusiveWrite() prevents this by:

1. Cancelling the TX coroutine job -- no more packets are queued for transmission
2. Cancelling the RX coroutine job -- no more packets are drained into the RX queue
3. Draining the TX and RX channels to discard in-flight packets
4. Waiting for both jobs to terminate with cancelAndJoin()
5. Performing the write and echo read directly on the USB connection object
6. Restarting the TX and RX workers in a finally block

This ensures the SG-100 response bus is silent and the first packet received after the write request is the FC16 echo.

---

5. Supported Write Operations

All writable holding registers use the same FC16 single-register write mechanism. The table below lists the writable registers in the SG-100 holding register block.

Register	Wire Offset	Label	Unit	Range
40051	0x0032	Overspeed Setting	RPM	0-4000
40052	0x0033	Start Fuel Position	%	0-100
40053	0x0034	Speed Ramp Time	s	0-100
40054	0x0035	Fuel Ramp Time	s	0-100
40055	0x0036	Crank Termination	RPM	0-2000
40056	0x0037	Speed Trim FS	RPM	0-6000
40057	0x0038	Speed Trim DS	RPM	0-6000
40058	0x0039	Idle Speed	RPM	0-3000
40059	0x003A	Speed 3	RPM	0-5000
40060	0x003B	Speed 2	RPM	0-5000
40061	0x003C	Speed 1	RPM	0-5000
40062	0x003D	Gear Teeth + Position Mode	packed	0-65535
40063	0x003E	Position PID D2	×10	0-1000
40064	0x003F	Position PID I2	×10	0-1000
40065	0x0040	Position PID P2	×10	0-1000
40066	0x0041	Position PID D1	×10	0-1000
40067	0x0042	Speed PID I1	×10	0-1000
40068	0x0043	Speed PID P1	×10	0-1000
40069	0x0044	Config Bits + Current Limit	packed	0-65535
40070	0x0045	PID Loop Time	ms	0-255
40071	0x0046	PWM Frequency + Droop	packed	0-65535
40072	0x0047	Full Load Current	×100 A	0-600
40074	0x0049	No Load Current	×100 A	0-600
40075	0x004A	Acceleration Time	×10 s	0-250
40076	0x004B	Deceleration Time	×10 s	0-250
40077	0x004C	Advanced Configuration	packed	0-65535

Register 40073 is explicitly marked non-writable in the register map. Register 40079 (wire offset 0x004E) is the write-enable gate and is written only by the internal write-enable sequence, not by user interaction.

---

6. Data Conversion Logic

6.1 Integer to Wire Format

All register values are unsigned 16-bit integers transmitted big-endian (high byte first). The application converts display values to raw wire integers before packet construction.

```
 raw_value = display_value * scale_factor

Wire encoding:
  data_high = (raw_value >> 8) & 0xFF
  data_low  =  raw_value       & 0xFF
```

6.2 Direct Integer Registers (Scale = 1)

Speed setpoints, ramp times, and timing values are written with no scaling.

```
 Speed 1 = 1400 RPM

raw_value = 1400
          = 0x0578

data_high = 0x05
data_low  = 0x78

Packet bytes 7-8: 05 78
```

6.3 Scaled Registers (×10 Storage)

PID gains and current values are stored multiplied by 10 or 100, allowing decimal precision to be represented as integers.

```
 PID registers (×10):
  Display value:  30.0
  raw_value:      300 = 0x012C
  data_high = 0x01
  data_low  = 0x2C
  Packet bytes 7-8: 01 2C

Current registers (×100):
  Display value:  2.50 A
  raw_value:      250 = 0x00FA
  data_high = 0x00
  data_low  = 0xFA
  Packet bytes 7-8: 00 FA
```

6.4 Packed Registers

Some registers contain multiple fields packed into a single 16-bit word. The

application must preserve the unmodified field when writing a packed register.

```
 Register 40062: Gear Teeth (bits 0-7) + Position Mode (bits 8-15)

Read current value: e.g. 0x0078 (position mode = 0, gear teeth = 120)
User changes gear teeth to 100:
  new_value = (position_mode << 8) | new_gear_teeth
            = (0x00 << 8) | 0x64
            = 0x0064

Register 40069: Config Bits (bits 8-15) + Current Limit % (bits 0-7)

Read current: 0x4646 (config bits = 0x46, current limit = 70%)
User changes current limit to 80%:
  new_value = (config_bits << 8) | new_limit
            = (0x46 << 8) | 0x50
            = 0x4650
```

The application should read the current value before writing packed registers to avoid corrupting the non-modified field.

6.5 Endianness Summary

```
 Data field         Byte order      Notes
Register values    Big-endian      High byte first in data payload
CRC16              Little-endian   Low byte first, high byte second
Register address   Big-endian      High byte first in address field
Register count     Big-endian      High byte first in count field
```

---

7. CRC / Checksum Handling

7.1 CRC16 Modbus Algorithm

The SG-100 uses the standard Modbus CRC16 algorithm:

```
 Polynomial:    0x8005 (reflected: 0xA001)
Initial value: 0xFFFF
Input/output:  reflected (LSB first)
Final XOR:     0x0000
```

The CRC is computed over all bytes in the frame except the CRC bytes themselves.
For an FC16 write request of 11 bytes total, the CRC covers bytes 0-8 (9 bytes).

7.2 CRC Calculation Example

```
 Frame (before CRC): 01 10 00 3C 00 01 02 05 78

Step-by-step CRC16 Modbus computation:
  Initial CRC = 0xFFFF

  Byte 0x01: CRC = 0xFFFF XOR 0x01 = 0xFFFE
             Shift 8 times with polynomial...  -> 0x8005 reduction
  ...
  (full table-lookup or bit-shift loop)
  ...
  Final CRC = 0xC546

  CRC_Low  = 0xC5
  CRC_High = 0x46

Complete packet: 01 10 00 3C 00 01 02 05 78 C5 46
```

Kotlin implementation:

```
 object Crc16 {
    fun appendTo(frame: ByteArray): ByteArray {
        var crc = 0xFFFF
        for (byte in frame) {
            crc = crc xor (byte.toInt() and 0xFF)
            repeat(8) {
                crc = if (crc and 0x0001 != 0) {
                    (crc ushr 1) xor 0xA001
                } else {
                    crc ushr 1
                }
            }
        }
        return frame + byteArrayOf(
            (crc and 0xFF).toByte(),
            ((crc ushr 8) and 0xFF).toByte(),
        )
    }
}
```

7.3 TX Packet CRC Verification

ModbusPacketBuilder computes and appends the CRC to every outgoing packet. Outgoing packet integrity is guaranteed; no CRC error is possible on the TX path.

7.4 RX Echo CRC Verification

RegisterDecoder.decode() verifies the CRC on every received packet:

```
 val crc = Crc16.verify(modbusFrame)
require(crc.ok) {
    "CRC mismatch received=${crc.received.hex16()} calculated=${crc.calculated.hex16()}"
}
```

A CRC mismatch on the echo causes an exception, which propagates to the writeSingleRegister() catch block and returns `WriteResult.Failure("CRC error: ...")`.

7.5 CRC Wire Position

CRC bytes are always the final two bytes of the Modbus RTU frame. They are not telemetry data. The low byte precedes the high byte (little-endian on the wire):

```
Frame tail: ... [data last] [CRC Low] [CRC High]
```

---

8. Firmware Interaction

8.1 Firmware Architecture

The SG-100 firmware executes a real-time PID control loop that continuously reads holding register values and applies them as control parameters. When the application writes a new value to a holding register, the firmware applies it to the next PID cycle immediately, without requiring a reset or explicit activation command.

```
SG-100 Firmware Control Loop (simplified)

 loop:
   speed_error = requested_speed - measured_speed
   p_term      = speed_error * holding[Speed_PID_P1]
   i_term      = integral * holding[Speed_PID_I1]
   d_term      = derivative * holding[Speed_PID_D1]
   output      = clamp(p_term + i_term + d_term, 0, current_limit)
   drive_actuator(output)
   wait(holding[PID_Loop_Time] ms)
```

8.2 Write-Enable Gate

The SG-100 firmware implements a write-enable gate on its holding register write path. A write to an arbitrary holding register is processed only if register 40079 currently holds the value 0x0080 (128). This prevents accidental configuration changes from noise or unintended Modbus traffic.

The desktop PC software sets this gate by writing 0x0080 to register 40079 before every configuration write sequence. The Android application replicates this behaviour:

```
 1. Write 0x0080 to register 40079 (wire offset 0x004E) via FC16
2. Wait 200 ms for the firmware to process the gate write
3. Write the target configuration register via FC16
4. Firmware processes the write because gate is open
```

The write-enable value (0x0080 = 128) was identified through IL bytecode disassembly of the Windows configuration tool binary at address 0x268FD:

```
 IL offset   Instruction     Value
0x26903     ldc.i4          128    <- write-enable value
0x26908     ldc.i4.s        78     <- register offset (40079)
0x2690A     newobj          WriteCommand(value, offset)
0x2690F     callvirt        EnqueueWrite()
```

8.3 USB Transport Layer

The SG-100 exposes a single USB HID interface with two interrupt endpoints:

```
 Endpoint    Direction   Type         Use
0x81        IN          Interrupt    Device -> Android (responses, echoes)
0x01        OUT         Interrupt    Android -> Device (requests, writes)
EP0         IN/OUT      Control      HID class requests (SET_REPORT, GET_REPORT)
```

Write packets are submitted to the device via one of two paths, tried in order:

1. **HID SET_REPORT** (controlTransfer, EP0) -- equivalent to Windows HidD_SetOutputReport. Preferred when available.
2. **Interrupt OUT bulkTransfer** -- fallback when the control transfer fails. Uses the detected preferredReportIdShape (with or without 0x00 prefix) that was learned from successful read operations.

Read responses and write echoes are received on the interrupt IN endpoint via bulkTransfer.

---

9. Error Handling

9.1 Write Echo Timeout

If the device does not respond to the FC16 write within the echo timeout (300 ms), the write path falls back to readback verification:

```
 Primary path:   FC16 write -> wait for echo (300 ms) -> verify offset -> Success
Fallback path:  FC16 write -> timeout -> FC03 readback -> compare stored value
```

The fallback does not distinguish between "write was accepted silently" and "write was ignored". The FC03 readback is the definitive verification. If the readback value matches the intended write value, the operation is reported as

WriteResult.SuccessWithHolding. If the readback returns a different value, the write is reported as succeeded but the UI displays the actual device value.

```
 is ModbusDecodeResult.RegisterBlock -> {
    val stored = decoded.block.value(address)
    packetLogger.message("WRITE", "Readback | addr=$address stored=$stored wrote=$value")
    WriteResult.SuccessWithHolding(decoded.block)
}
```

9.2 CRC Mismatch on Echo

A CRC mismatch in the echo response throws an exception inside RegisterDecoder.decode(). This is caught by the exclusiveWrite() caller:

```
 } catch (e: Exception) {
    packetLogger.message("WRITE", "Write echo error: ${e.message} -- will readback")
}
```

The code proceeds to the FC03 readback path. A CRC error on the echo is treated the same as a timeout: the readback is the authoritative verification.

9.3 USB Transmission Failure

If both the control transfer and the bulk transfer fail (both return negative), exclusiveWrite() throws "HID write failed: ctrl=N bulk=N". This propagates to writeSingleRegister() and is caught:

```
 } catch (e: Exception) {
    WriteResult.Failure(e.message ?: "Unexpected write error")
}
```

The UI receives WriteResult.Failure and SettingsManager.markError() is called, displaying the error message next to the affected register field.

9.4 Device Disconnect During Write

If the USB device is disconnected during a write operation, exclusiveWrite() will receive an error from bulkTransfer() or the subsequent read loop will time out. The finally block in exclusiveWrite() always restarts the background workers, even if they immediately detect the disconnected state and terminate. The PollingManager.resume() call in the ViewModel ensures the background polling state machine returns to a clean state.

9.5 Write-Enable Failure

If the write-enable gate write (to register 40079) does not echo:

```
 } catch (e: Exception) {
    packetLogger.message("WRITE", "Write-enable no echo: ${e.message} -- proceeding with main write")
}
```

The main write is attempted anyway. The write-enable echo is not mandatory for the write sequence to proceed. If the gate was already open from a recent operation, the main write may succeed without a fresh write-enable.

9.6 Error State Summary

```
 Condition                          Result                    UI Behaviour
-------------------------------------------------------------------------------
Echo received, offset matches       WriteResult.Success       Green "OK" badge
No echo, readback matches           WriteResult.SuccessWithHolding  Green "OK"
No echo, USB transmission failure   WriteResult.Failure       Red error message
CRC error on echo, readback OK      WriteResult.SuccessWithHolding  Green "OK"
Device disconnected during write    WriteResult.Failure       Red error message
Write-enable fails, main fails      WriteResult.Failure       Red error message
```

---

10. Auto-Reconnect Architecture

10.1 Connection State Machine

The application maintains a continuous connection monitoring loop that operates independently of user interaction. The connection lifecycle is fully automatic.

```
States:

 SEARCHING ------------------------------------------------+
     |                                                     |
     |  Device plugged in (USB_DEVICE_ATTACHED broadcast)  |
     ?                                                     |
 CONNECTING                                               |
     |                                                     |
     |  USB permission granted                             |
     ?                                                     |
 CONNECTED                                                |
     |  Auto: load holding registers                       |
     |  Auto: start 10 Hz polling                          |
     |                                                     |
     |  Device unplugged                                   |
     ?                                                     |
 RECONNECTING ----------------------------------------->+
     |  Stop polling workers
     |  Retry connectBestDevice() every 3 seconds
```

10.2 USB Attach Event Handling

Android delivers a USB_DEVICE_ATTACHED broadcast intent when a USB device is connected to the OTG port. UsbHidManager registers a BroadcastReceiver for this intent at initialisation time.

```
 USB_DEVICE_ATTACHED intent received
    |
    ?
UsbHidManager.connectBestDevice()
    |
    +-- findBestDevice() -> match by VID/PID 0x04D8:0xF1BB
    |
    +-- usbManager.hasPermission(device)?
    |     YES -> openDevice()
    |     NO  -> requestPermission() -> wait for user approval
    |
    +-- openDevice()
          -> claimInterface()
          -> selectEndpoints()
          -> startWorkers()
          -> UsbHidState.connected = true
```

10.3 Reconnect Loop

DashboardViewModel maintains a coroutine-based reconnect loop that retries the connection every 3 seconds when the device is not connected:

```
 private fun startReconnectLoop() {
    reconnectJob?.cancel()
    reconnectJob = viewModelScope.launch {
        while (isActive) {
            delay(RECONNECT_INTERVAL_MS)        // 3000 ms
            val s = usbHidManager.state.value
            if (!s.connected && !s.permissionPending) {
                usbHidManager.connectBestDevice()
            }
        }
    }
}
```

The loop is started on app launch and restarted whenever a disconnect is detected. It is cancelled when the connection is established.

10.4 Auto-Start Telemetry on Reconnect

When UsbHidState.connected transitions from false to true, the ViewModel automatically loads the holding register block and starts the polling manager:

```
 .collect { connected ->
    if (connected) {
        reconnectJob?.cancel()
        loadHoldingRegisters()
        pollingManager.start()
    } else {
        pollingManager.stop()
        startReconnectLoop()
    }
}
```

This means the Configure tab is automatically populated with current device values immediately after every connection -- including reconnections after a cable re-plug.

---

11. UI Integration

11.1 Connection Status Display

The application header shows a live AutoStatusBar component that reflects the current connection and telemetry state. No manual interaction is required to initiate or restart the connection.

```
 State                          AutoStatusBar Display
-----------------------------------------------------------------
Not connected (searching)       * Searching for SG100?         (grey, pulsing)
Permission requested            * Tap OK to allow USB access    (amber, pulsing)
Connected, loading              * SG100 linked -- starting?      (amber, pulsing)
Connected, polling active       * Connected to SG100 · 10.0 Hz  (green, solid)
```

The dot size and background opacity pulse continuously when the app is in a transient state (searching, loading). They are steady when fully connected.

11.2 Configure Tab -- Register Write Flow

```
 Configure Tab Write Flow

1. User sees register current value (loaded from last FC03 read)
2. User adjusts slider or numeric field
   -> SettingsManager.edit(address, value) called
   -> Register marked dirty
   -> Write button becomes active (blue)
3. User taps Write
   -> Register marked pending
   -> Write button shows "?"
4. Write completes:
   Success:
     -> Register marked clean
     -> Write button shows "v" (green) for 2 seconds
     -> Displayed value updates to confirmed device value
   Failure:
     -> Register marked error
     -> Write button shows "x" (red) with error message
     -> User can retry
```

11.3 Register Display Synchronisation

After a successful write with readback (WriteResult.SuccessWithHolding), the application calls SettingsManager.loadHoldingRegisters(block.registers), which refreshes all 27 holding registers in the Configure tab from the device's current values. This ensures the UI reflects the actual device state, not just the intended write value.

```
 is WriteResult.SuccessWithHolding -> {
    val actual = result.block.value(address)
    settingsManager.loadHoldingRegisters(result.block.registers)
    settingsManager.markClean(address, actual)
}
```

11.4 Write Status Lifecycle

```
 WriteStatus enum:

  Idle     -> register is clean, no pending write
  Pending  -> write in progress (disable Write button, show "?")
  Success  -> write confirmed (green indicator, 2-second display)
  Error    -> write failed (red indicator, error text shown)

Transitions:
  Idle -> Pending   : markPending(address) called
  Pending -> Success: markClean(address, actual) after echo verify or readback
  Pending -> Error  : markError(address, reason) on failure
  Success -> Idle   : clearWriteStatus(address) after badge timeout
  Error -> Idle     : clearWriteStatus(address) on user dismiss or next edit
```

---

12. Logging and Diagnostics

12.1 TX Packet Logging

Every outgoing packet is logged to the PacketLogger with a human-readable label and a hex dump:

```
 TX  FC16 Write-enable | addr=40079 value=0x80
    01 10 00 4E 00 01 02 00 80 4C 03

TX  FC16 Write | addr=40061 offset=0x003C value=1400 (0x0578)
    01 10 00 3C 00 01 02 05 78 C5 46
```

12.2 RX Echo Logging

The echo and readback responses are logged:

```
 RX  Write-enable echo
    01 10 00 4E 00 01 15 C3

RX  FC16 Echo RX
    01 10 00 3C 00 01 A1 C8

INFO  WRITE  ACK verified | FC10 offset=0x003C value=1400
```

Or on fallback:

```
 INFO  WRITE  Write echo error: No HID response within 300ms (ctrl=fail) -- will readback
TX    FC03 Post-write readback | addr=40061
      01 03 00 32 00 1B A4 0E
RX    FC03 Readback RX
      01 03 36 ... [27 register values] ... CRC
INFO  WRITE  Readback | addr=40061 stored=1400 wrote=1400
```

12.3 Diagnostics Tab

The Diagnostics tab in the application UI displays the full chronological PacketLogger log. Each entry shows:

```
 HH:MM:SS.mmm   TYPE   LABEL
                      [hex dump if packet entry]
```

Entry types:

```
 TX    Outgoing Modbus RTU frame (hex dump included)
RX    Incoming Modbus RTU frame (hex dump included)
INFO  State machine event (no hex dump)
```

This log is sufficient for debugging protocol-level issues, timing problems, and CRC errors without requiring a physical USB analyser.

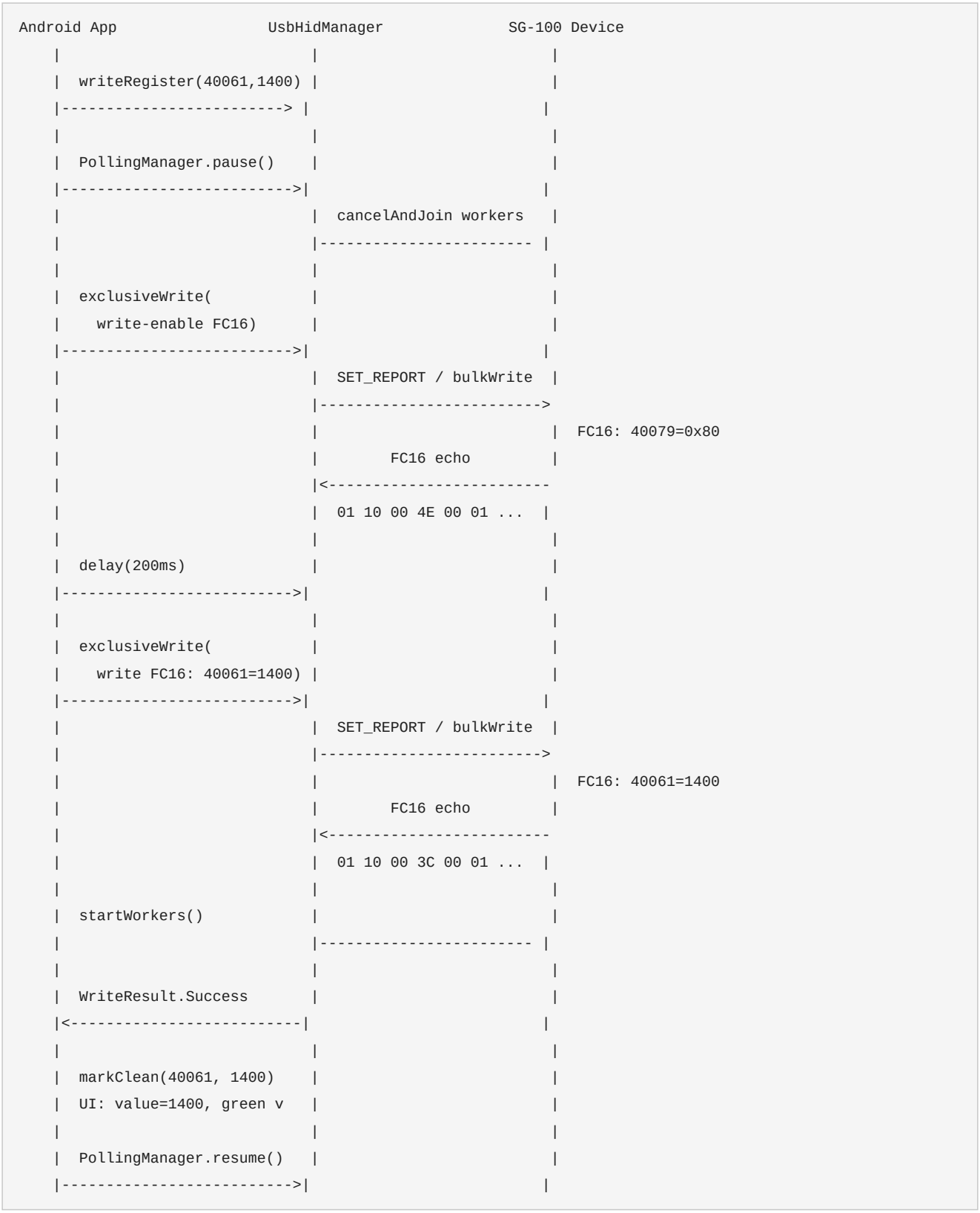
12.4 Key Diagnostic Messages

```
 Message                                          Meaning
-------------------------------------------------------------------------
"Workers stopping -- exclusive bus access"         Write sequence beginning
"Write-enable no echo: ... -- proceeding"          Gate write not echoed; continuing
"ACK verified | FC10 offset=0x003C value=1400"    Write confirmed by echo
"Write echo error: No HID response -- will readback"  Echo timed out; using readback
"Readback | addr=40061 stored=1400 wrote=1400"    Readback confirmed write
"Readback | addr=40061 stored=1500 wrote=1400"    Write failed; value unchanged
"FAIL 40061: CRC error: ..."                       Echo had invalid CRC
```

---

13. Sequence Diagrams

13.1 Successful Write with Echo Confirmation



13.2 Write with Readback Fallback (No Echo)

13.3 Auto-Reconnect Flow

```
Android App             UsbHidManager           Android OS / SG-100
    |                       |                       |
    |  [device unplugged]   |                       |
    |                       |  USB_DEVICE_DETACHED  |
    |                       |<----------------------
    |                       |  close()              |
    |                       |  state.connected=false |
    |                       |                       |
    |  pollingManager.stop() |                       |
    |  startReconnectLoop()  |                       |
    |                       |                       |
    |  [every 3 seconds]    |                       |
    |  connectBestDevice() ->|                       |
    |                       |  findBestDevice()=null |
    |                       |  state.msg=Searching  |
    |  AutoStatusBar:       |                       |
    |  "Searching for SG100" |                       |
    |                       |                       |
    |  [device plugged in]  |                       |
    |                       |  USB_DEVICE_ATTACHED  |
    |                       |<----------------------
    |                       |  connectBestDevice()  |
    |                       |  openDevice()         |
    |                       |  startWorkers()       |
    |                       |  state.connected=true |
    |                       |                       |
    |  reconnectJob.cancel() |                       |
    |  loadHoldingRegisters()|                       |
    |  pollingManager.start()|                       |
    |  AutoStatusBar:       |                       |
    |  "Connected · 10.0 Hz" |                       |
```

---

14. Future Improvements

14.1 Batch Register Writing

The current implementation writes one register at a time, requiring a separate write-enable and FC16 transaction per parameter. The Modbus FC16 function code supports writing up to 123 consecutive registers in a single packet. A batch write operation could write multiple modified registers atomically, reducing the total number of round-trips and the risk of partial configuration updates.

```
Proposed batch write packet (3 registers starting at 40063):
 01 10 00 3E 00 03 06 [D2_H][D2_L][I2_H][I2_L][P2_H][P2_L] CRC_L CRC_H
```

14.2 Write Queue

When the user modifies multiple registers in rapid succession, each write

currently runs sequentially with a full exclusive-write window. A write queue would allow modifications to be coalesced or batched:

* Coalesce multiple writes to the same register (keep only the latest value)
* Batch adjacent-address writes into a single FC16 multi-register transaction
* Enforce a minimum inter-write delay to protect the SG-100 firmware state machine

14.3 Asynchronous Write-Enable Cache

The write-enable gate register (40079) has an unknown firmware timeout after which it resets. If the timeout is measurable (e.g., 1-5 seconds), the application could cache the write-enable state and skip the gate write when a recent write-enable is still likely active, reducing write latency by 200-400 ms.

14.4 Expanded Register Coverage

The current holding register read covers 27 registers (40051-40077). The PC configuration tool reads 31 registers (40051-40081), including registers 40078-40081 that are not yet mapped in the application. Expanding the register read range would expose additional configuration parameters and improve the completeness of the readback-after-write verification.

14.5 Write Verification Strictness

The current readback path returns WriteResult.SuccessWithHolding regardless of whether the stored value matches the intended value. A strict verification mode could compare stored against wrote and return WriteResult.Failure if they differ, providing definitive confirmation that the SG-100 firmware rejected or ignored the write.

14.6 Firmware Compatibility Layer

The write-enable register address and value were determined by reverse-engineering a specific firmware version of the PC configuration tool. Future firmware updates may change this protocol. A firmware-version-aware write path could select the appropriate write-enable mechanism based on the firmware version read from input register 30062 at connection time.

14.7 Improved Diagnostics Export

The current PacketLogger stores entries in memory and displays them in the Diagnostics tab. A future improvement could export the packet log to a file (CSV or plain text) for off-device analysis, or expose a live WebSocket stream for use with protocol analyser tools.

Appendix A -- Key Constants

Constant	Value	Description
WRITE_ECHO_TIMEOUT_MS	300	Maximum wait for FC16 echo (ms)
WRITE_SETTLE_MS	200	Delay after write-enable or echo
WRITE_ENABLE_REGISTER	40079	Write-enable gate register address
WRITE_ENABLE_VALUE	0x80 (128)	Value that opens the write gate
HOLDING_START	40051	First holding register address
HOLDING_COUNT	27	Number of holding registers read
HID_REPORT_LENGTH	64	Fixed USB HID report size (bytes)
WRITE_TIMEOUT_MS	250	USB transfer timeout (ms)
RECONNECT_INTERVAL_MS	3000	Reconnect loop retry interval (ms)
SG100_VENDOR_ID	0x04D8	Microchip USB vendor ID
SG100_PRODUCT_ID	0xF1BB	SG-100 product ID

Appendix B -- Modbus Function Codes Used

Code	Hex	Name	Direction	Used For
FC03	0x03	Read Holding Registers	Read	Config register readback
FC04	0x04	Read Input Registers	Read	Live telemetry polling
FC16	0x10	Write Multiple Registers	Write	All configuration writes
FC06	0x06	Write Single Register	(unused)	Rejected by SG-100 firmware

Appendix C -- Register Address Quick Reference

Write Operation	Register	Offset	Example Value	Raw Wire
Write-enable gate	40079	0x004E	0x0080	00 80
Speed 1	40061	0x003C	1400 RPM	05 78
Speed 2	40060	0x003B	1400 RPM	05 78
Speed 3	40059	0x003A	1400 RPM	05 78
Idle Speed	40058	0x0039	800 RPM	03 20
Overspeed	40051	0x0032	2000 RPM	07 D0
Speed PID P1	40068	0x0043	50.0 (raw 500)	01 F4
Speed PID I1	40067	0x0042	10.0 (raw 100)	00 64
Position PID P2	40065	0x0040	50.0 (raw 500)	01 F4
Position PID I2	40064	0x003F	10.0 (raw 100)	00 64
Position PID D2	40063	0x003E	30.0 (raw 300)	01 2C
PID Loop Time	40070	0x0045	10 ms	00 0A